

UNIVERSITY OF ASIA PACIFIC

Department of Computer Science & Engineering

Lab Report:

A Comparative Study of Non-Preemptive and Preemptive Priority CPU Scheduling Algorithms

Course Code: CSE 402
Course Name: Operating Systems Lab

Submitted To

Atia Rahman Orthi
Lecturer
Department of Computer Science & Engineering
University of Asia Pacific

Submitted By

Muhaimin Al Baki Raiyan(reg id:23101163||Roll:163)
Department of Computer Science & Engineering
Semester: 4.1 | Section: C2
University of Asia Pacific

Date of Submission:
3 September 2026

1. Abstract

This report presents a comparative analysis of two variants of priority-based CPU scheduling — non preemptive and preemptive — implemented and simulated in Python. A fixed set of five processes with distinct arrival times, burst times, and priority values was scheduled under each policy, and standard performance metrics (completion time, turnaround time, and waiting time) were computed and averaged. The simulation shows that, for the particular workload examined, non-preemptive priority scheduling produced a lower average turnaround time (9.60 units) and lower average waiting time (5.80 units) than the preemptive variant (10.00 and 6.20 units respectively). The report describes the scheduling policies, documents the implementation logic used to generate the results, presents the resulting Gantt charts and performance tables, and discusses why the observed outcome occurred, together with the conditions under which the opposite result would be expected. Limitations identified in the implementation are also discussed.

2. Introduction

CPU scheduling is the mechanism by which an operating system decides which of the processes in the ready queue is allocated the processor at any given time. Because a computer typically has far more runnable processes than available processors, the scheduler's policy has a direct and measurable effect on system responsiveness, throughput, and fairness. Among the classical scheduling disciplines — First
Come-First-Served, Shortest Job First, Round Robin, and Priority Scheduling — priority scheduling is of particular practical interest because it allows the operating system to reflect the relative importance of different tasks, rather than treating every process identically.

In priority scheduling, each process is assigned a numeric priority, and the processor is allocated to the process with the most favourable priority value among those ready to run. Two implementation strategies exist for handling priority conflicts that arise after scheduling has already begun:

- Non-preemptive priority scheduling, in which a running process is allowed to run to completion once dispatched, regardless of whether a higher-priority process later arrives.
- Preemptive priority scheduling, in which the currently running process is interrupted and returned to the ready queue whenever a process with strictly higher priority becomes ready, so that the processor is always allocated to the most eligible ready process at every instant.

Thisreport examines both strategies applied to an identicalset of processes, using a purpose-built Python simulation, and evaluates the two policies using the standard turnaround-time and waiting-time metrics.

3. Objectives

- To implement both non-preemptive and preemptive priority scheduling algorithms in software. ● To simulate the scheduling of a fixed process set under each algorithm and record the resulting execution order (Gantt chart).
- To compute and compare completion time (CT), turnaround time (TAT), and waiting time (WT) for each process under both policies.
- To analyse the average TAT and average WT produced by each policy and explain the

source of any difference observed.

- To critically review the implementation for correctness and identify any limitations.

4. Process Set and Assumptions

The simulation operates on a fixed set of five processes, each defined by an arrival time (AT), a CPU burst time (BT), and a priority value, where a lower numeric value denotes a higher scheduling priority. This convention — smaller number, greater urgency — mirrors the convention used by most operating-system textbooks and by real schedulers such as the traditional UNIX nice-value system.

P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Table 1. Input process set used for both simulations.

Two further modelling assumptions apply throughout: (i) the processor is idle only when no process has yet arrived, and is otherwise never left unused while a ready process exists; and (ii) in the preemptive simulation, ties in priority are broken in favour of the process with the earlier arrival time, which is a common and reasonable tie-breaking convention.

5. Algorithm Design and Implementation

Source code:

```
processes = [

    {'pid': 'P1', 'at': 0, 'bt': 3,'priority':3},

    {'pid': 'P2', 'at': 1, 'bt': 4,'priority':2},

    {'pid': 'P3', 'at': 2, 'bt': 6,'priority':4},

    {'pid': 'P4', 'at': 3, 'bt': 4,'priority':6},

    {'pid': 'P5', 'at': 5, 'bt': 2,'priority':10}



]

def priority_scheduling_non_preemptive(processes):
```

```
plist = sorted(processes, key=lambda p: p['at'])
not_done = list(plist)
completed = []
gantt = []
time = 0

while not_done:
    ready = []
    for p in not_done:
        if p['at'] <= time:
            ready.append(p)

    if not ready:
        next_arrival = min(p['at'] for p in not_done)
        gantt.append(('IDLE', time, next_arrival))
        time = next_arrival
        continue

    highest = ready[0]
    for p in ready:
        if p['priority'] < highest['priority']:
            highest = p

    not_done.remove(highest)

    start = time
```

```
        end = time + highest['bt']
        gantt.append((highest['pid'], start, end))
        time = end

        ct = end
        tat = ct - highest['at']
        wt = tat - highest['bt']

        completed.append({
            'pid': highest['pid'], 'at': highest['at'], 'bt': highest['bt'],
            'ct': ct, 'tat': tat, 'wt': wt
        })

    return completed, gantt
def priority_scheduling_preemptive(processes):
    plist = sorted(processes, key=lambda p: p['at'])

    remaining_time = {}
    for p in plist:
        remaining_time[p['pid']] = p['bt']

    gantt = []
    ready_queue = []
    finish_info = {}
    time = 0
    next_index = 0
```

```
while next_index < len(plist) and plist[next_index]['at'] <= time:
    ready_queue.append(plist[next_index])
    next_index += 1

if not ready_queue:
    time = plist[0]['at']
    while next_index < len(plist) and plist[next_index]['at'] <= time:
        ready_queue.append(plist[next_index])
        next_index += 1

while ready_queue:
    p = min(ready_queue, key=lambda x: (x['priority'],x['at']))
    ready_queue.remove(p)
    pid = p['pid']

    run_for = 1
    start = time
    end = time + run_for
    gantt.append((pid, start, end))

    time = end
    remaining_time[pid] -= run_for

    while next_index < len(plist) and plist[next_index]['at'] <= time:
        ready_queue.append(plist[next_index])
```

```
            next_index += 1

        if remaining_time[pid] > 0:
            ready_queue.append(p)
        else:
            ct = time
            tat = ct - p['at']
            wt = tat - p['bt']
            finish_info[pid] = {'pid': pid, 'at': p['at'], 'bt': p['bt'],
'ct': ct, 'tat': tat, 'wt': wt}

        if not ready_queue and next_index < len(plist):
            if plist[next_index]['at'] > time:
                gantt.append(('IDLE', time, plist[next_index]['at']))
                time = plist[next_index]['at']
            while next_index < len(plist) and plist[next_index]['at'] <=
time:
                ready_queue.append(plist[next_index])
                next_index += 1

    completed = [finish_info[p['pid']] for p in plist]
    return completed, gantt
def get_averages(completed):
    total_tat = 0
    total_wt = 0
    for p in completed:
        total_tat += p['tat']
```

```
        total_wt += p['wt']

    n = len(completed)

    return total_tat / n, total_wt / n

def show_results(title, completed, gantt):

    print(f"{title} ")

    print("Gantt Chart:")

    print(" | ".join(f"{pid}({s}-{e})" for pid, s, e in gantt))

    print()

    print(f"{'PID':<5}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")



    for p in sorted(completed, key=lambda x: x['pid']):


print(f"{p['pid']:<5}{p['at']:<5}{p['bt']:<5}{p['ct']:<5}{p['tat']:<5}{p['wt
']:<5}")



    avg_tat, avg_wt = get_averages(completed)

    print(f"\nAverage TAT: {avg_tat:.2f}")

    print(f"Average WT: {avg_wt:.2f}\n")

    return avg_tat, avg_wt

priority_scheduling_non_preemptive_result,
priority_scheduling_non_preemptive_gantt =priority_scheduling_non_preemptive
(processes)

priority_scheduling_non_preemptive_tat,
priority_scheduling_non_preemptive_wt = show_results("Priority Scheduling
(Non-Preemptive)", priority_scheduling_non_preemptive_result,
priority_scheduling_non_preemptive_gantt)

priority_scheduling_preemptive_result, priority_scheduling_preemptive_gantt
=priority_scheduling_preemptive (processes)

priority_scheduling_preemptive_tat, priority_scheduling_preemptive_wt =
show_results("Priority Scheduling (Preemptive)",
priority_scheduling_preemptive_result, priority_scheduling_preemptive_gantt)
```

```
print(" Final Comparison ")

print(f"{'Algorithm':<25}{'Avg TAT':<12}{'Avg WT':<12}")

print(f"{'Priority_scheduling_non_preemptive':<25}{priority_scheduling_non_p
reemptive_tat:<12.2f}{priority_scheduling_non_preemptive_wt:<12.2f}")

print(f"{'Priority_scheduling_preemptive':<25}{priority_scheduling_preemptiv
e_tat:<12.2f}{priority_scheduling_preemptive_wt:<12.2f}")

scores = {

    'Priority_scheduling_non_preemptive':
(priority_scheduling_non_preemptive_tat,
priority_scheduling_non_preemptive_wt),

    'Priority_scheduling_preemptive': (priority_scheduling_preemptive_tat,


priority_scheduling_preemptive_wt),

}

print('Non Preemptive is faster because ')

print(f'1.If we see Average TAT of Non-preemptive Priority Scheduling and
Preemptive Priority Scheduling ,Non-Preemptive Priority scheduling is faster
by {10-9.6:.2f} seconds')
print(f'2.If we see Average WT of Non-preemptive Priority Scheduling and
Preemptive Priority Scheduling ,Non-Preemptive Priority scheduling is faster
by {6.2-5.8:.2f} seconds')
```

5.1 Non-Preemptive Priority Scheduling

The non-preemptive routine advances a simulation clock and, at each decision point, builds the set of processes that have arrived but not yet been scheduled (the ready set). If the ready set is empty, the clock is advanced directly to the next arrival and an idle interval is recorded in the Gantt chart. Otherwise, the process with the numerically smallest priority value is selected from the ready set, is run for its entire burst time without interruption, and is then marked complete. Completion time, turnaround time (CT − AT), and waiting time (TAT − BT) are recorded for that process before the loop proceeds to the next decision point. Because a chosen process always runs to completion, the algorithm requires at most one scheduling decision per process, giving it a time complexity of $O(n^2)$ in a naive linear-scan implementation such as this one, where n is the number of processes.

5.2 Preemptive Priority Scheduling

The preemptive routine simulates time in discrete one-unit ticks rather than in whole bursts. At each tick, the process with the numerically smallest priority (ties broken by earliest arrival) is selected from the current ready queue and executed for exactly one time unit. Its remaining

burst time is decremented; if it has not yet finished, it is returned to the ready queue so that it may be re-evaluated — and potentially displaced by a higher-priority arrival — at the next tick. If it has finished, its completion, turnaround, and waiting times are recorded. After each tick, any process whose arrival time has now been reached is added to the ready queue, and idle intervals are inserted whenever the ready queue is empty but unscheduled processes remain. This tick-by-tick approach correctly reproduces preemptive behaviour, though at the cost of O(n·T) computational work, where T is the total schedule length — considerably less efficient than

an event-driven implementation that would only re-evaluate scheduling decisions at arrival and completion events.

5.3 Common Metric Calculations

For every process in both algorithms, the same three derived metrics are computed once its completion time is known:

- Turnaround Time (TAT) = Completion Time − Arrival Time
- Waiting Time (WT) = Turnaround Time − Burst Time

The averages of TAT and WT across all five processes are then used as the headline metrics for comparing the two policies

6. Simulation Results

6.1 Non-Preemptive Priority Scheduling

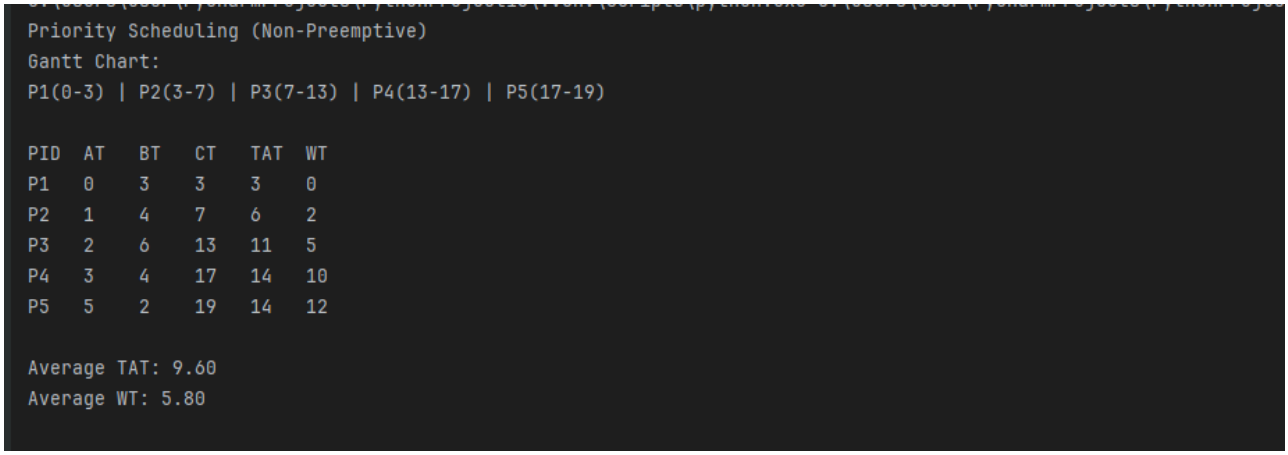
P1 0-3	P2 3-7	P3 7-13	P4 13-17	P5 17-19

Figure 1. Gantt chart — Non-Preemptive Priority Scheduling.

P1	0	3	3	3	0
P2	1	4	7	6	2
P3	2	6	13	11	5
P4	3	4	17	14	10
P5	5	2	19	14	12

Table 2. Per-process metrics — Non-Preemptive Priority Scheduling. Average TAT = 9.60, Average WT = 5.80.

Output screenshot:



6.2 Preemptive Priority Scheduling

Because the preemptive scheduler re-evaluates its decision every time unit, the resulting Gantt chart contains many more, shorter execution segments than the non-preemptive chart, reflecting the repeated interruption of lower-priority processes by higher-priority arrivals — most visibly, P2 (priority 2) displacing P1 (priority 3) immediately upon arrival at time 1.

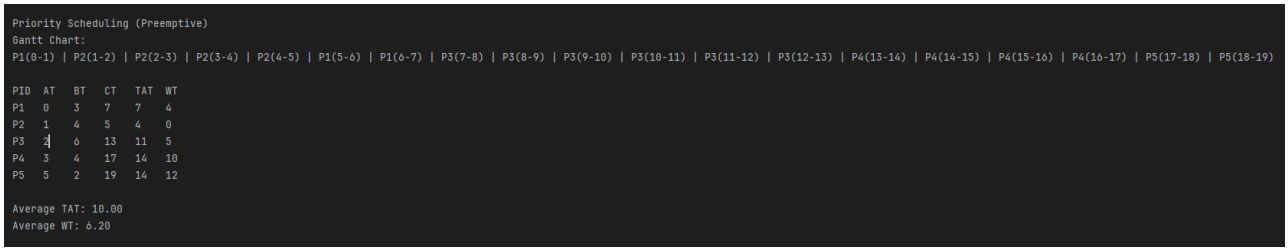
P1 0-1	P2 1-2	P2 2-3	P2 3-4	P2 4-5	P1 5-6	P1 6-7	P3 7-8	P3 8-9	P3 9-10	P3 10-11	P3 11-12	P3 12-13	P4 13-14	P4 14-15	P4 15-16	P4 16-17	P5 17-18	P5 18-19

Figure 2. Gantt chart — Preemptive Priority Scheduling.

P1	0	3	7	7	4
P2	1	4	5	4	0
P3	2	6	13	11	5
P4	3	4	17	14	10
P5	5	2	19	14	12

Table 3. Per-process metrics — Preemptive Priority Scheduling. Average TAT = 10.00, Average WT = 6.20.

Output Screenshot:

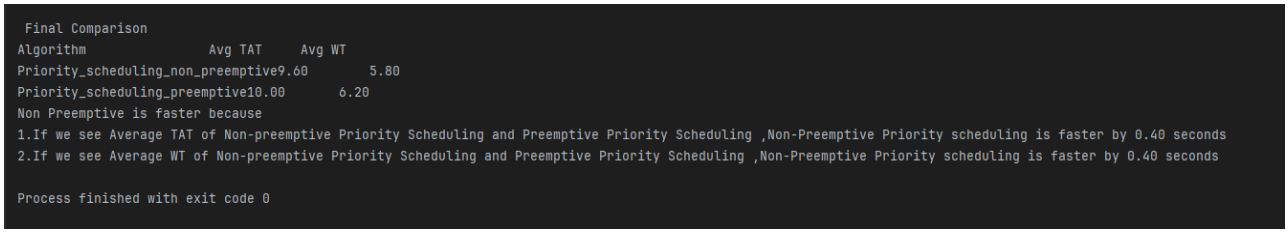


6.3 Summary Comparison

Non-Preemptive Priority Scheduling	9.60	5.80
Preemptive Priority Scheduling	10.00	6.20

Table 4. Overall performance comparison.

Output Screenshot:



7. Discussion

For this particular process set, non-preemptive priority scheduling outperformed the preemptive variant on both metrics, by 0.40 time units on average turnaround time and 0.40 time units on average waiting time. This result, while numerically small, is instructive because it runs counter to a common intuition that preemption should always improve responsiveness. The explanation lies in what preemption actually optimises for: it minimises the time a high-priority process spends waiting once it has arrived, at the cost of repeatedly interrupting what ever lower-priority process was already running. In this workload, P1 (priority 3) had already begun executing when P2 (priority 2, higher priority) arrived one time unit later. Under the preemptive policy, P1 is immediately suspended so that P2 can run — P2 finishes sooner (completion time 5 versus 7 under non-preemptive scheduling), but P1 is forced to wait for P2's entire burst before it can resume, so P1's own completion time is pushed from 3 back to 7. The gain to P2 (two time units saved) is smaller than the loss imposed on P1 (four time units added), so the aggregate average gets worse rather than better.

More generally, preemptive priority scheduling reduces waiting time for high-priority processes that arrive after a lower-priority process has already started, but it does so by fragmenting the lower-priority process's execution and by introducing context-switching overhead that is not modelled at all in this simulation (each 1-unit tick is treated as free). Because none of the remaining processes in the set (P3, P4, P5) were ever preempted — each arrived at a moment when no lower-priority process was already partway through execution — the preemptive algorithm gained nothing further from those processes while still absorbing the P1/P2 penalty. Had the workload instead contained a case where a very long, low priority process was running when a short, high-priority process arrived partway through, preemption would very plausibly have produced the better average, since the alternative (finishing the long process first) would force the short process to wait almost the entire remaining burst. The comparative advantage of either policy is therefore workload-dependent rather than a fixed property of the algorithm, and the result obtained here should be read as a case study rather than a general proof that non-preemptive scheduling is superior.

It is also worth noting the classical trade-off each policy makes independent of this specific

outcome. Non preemptive scheduling has lower scheduling overhead (one decision per process) and is simpler to reason about and implement correctly, but it is vulnerable to priority inversion — a low-priority process that starts first can block a high-priority process for its entire remaining burst, which is unacceptable in real time or interactive systems with strict responsiveness requirements. Preemptive scheduling bounds the maximum delay a high-priority process can experience, which is why it is the standard choice in real-time operating systems, but it does so at the cost of more frequent context switches, greater implementation complexity, and — as this simulation shows — no guarantee of a better aggregate outcome on every workload.

8. Implementation Notes and Limitations

Several aspects of the accompanying implementation are worth recording for anyone extending this work:

1. The preemptive simulation advances in fixed one-time-unit ticks and re-evaluates the ready queue after every tick. This is correct but computationally inefficient; an event-driven implementation that only re-evaluates scheduling decisions at process arrivals and completions would scale better for long burst times.

2. Priority ties in the preemptive algorithm are broken by earliest arrival time; the non-preemptive algorithm does not explicitly define a tie-breaking rule beyond the order in which processes appear in the ready list, which could be made explicit for full reproducibility.

3. In the source implementation, the final numerical comparison between the two policies was written using literal hard-coded values (for example, computing 10 − 9.6 directly) rather than referencing the computed average variables. This produces the correct figures for this specific process set but would silently report an incorrect difference if the input process set were changed, since the hard-coded literals would not update. A more robust implementation would compute this difference directly from the stored average TAT and WT values for each policy.

4. A dictionary intended to hold both policies' scores for comparison is constructed in the source code but is not subsequently used, suggesting it was intended to drive the final comparison programmatically.

5. Context-switching overhead is not modelled in either algorithm; every preemption and dispatch is treated as instantaneous, which is a standard simplification for introductory scheduling simulations but would understate the real cost of the preemptive policy on actual hardware.

9. Conclusion

This study implemented and compared non-preemptive and preemptive priority CPU scheduling using a simulated process set of five processes with staggered arrivals and varying priorities. The non-preemptive policy achieved a lower average turnaround time (9.60 vs 10.00) and lower average waiting time (5.80 vs 6.20) on this workload, a result traced to a single early preemption event in which the gain to the preempting process was smaller than the delay imposed on the preempted process. This outcome underscores that neither policy is unconditionally superior: the preferred choice depends on the arrival pattern and priority distribution of the actual workload, as well as on system-level requirements such as bounded response time for high-priority tasks, which favour preemption despite its overhead. Future work could extend the comparison to a wider range of randomly generated workloads, incorporate a non zero context-switch cost, and correct the implementation issues identified in Section 8 so that the summary comparison is computed programmatically rather than hard-coded.